

The project in plain words - with the technique used at each step

One page, start to finish. Left column: what happens. Right: the actual method.

The problem, in two sentences

Teaching a robot household tasks normally needs a human physically guiding the arm hundreds of times. We film a person instead - but then the robot learns everything from camera footage, and cameras have bad moments, so the robot ends up confidently acting on things it cannot actually see.

Step 1 - Record people doing tasks

What: Film a person performing five household tasks, plus a "doing nothing" state that turns out to matter a lot.

Technique	RGB-D camera (colour + depth), 640x400 at 15 fps
Tasks	Bottle, Clock, Fan, Plant, Wave, + Rest
Scale	906 usable trials, collected in six condition blocks per task

Step 2 - Find the hand in 3D

What: For every frame, work out where the hand is and how it's shaped.

Technique	WiLoR-mini (HaMeR-family, MANO hand model)
Output	21 joint positions + a 778-vertex hand mesh

Step 3 - Separate hand, arm and object from the background

What: Cut out which pixels are hand, which are arm, which are the object.

Technique	SAM2 (Segment Anything 2) for masks + object tracking
-----------	---

Step 4 - Sharpen the hand position using depth

What: The vision estimate is close but not exact. Nudge it until it lines up with what the depth sensor actually measured.

Technique	ICP (Iterative Closest Point) - register the hand mesh against the depth point cloud
Bonus	ICP's fit quality later becomes a signal for <i>how much to trust that frame</i>

Step 5 - Translate human motion into robot motion

What: A human hand and a robot gripper aren't the same shape. Convert one to the other.

Technique	Cartesian retargeting - hand pose -> end-effector position, orientation, gripper opening
Output	Actions already in the robot's own coordinate frame
Note	~23% of frames have no computable action; those are smoothly interpolated

Step 6 - Erase the human from the video

What: The robot shouldn't be trained on footage with a person's arm in it.

Technique	Background-plate reconstruction using SAM2's arm mask (E2FGVI-family video inpainting)
-----------	--

Step 7 - Draw the robot doing the task

What: Render a simulated Kinova arm performing the same motion.

Technique	MuJoCo physics simulation, Kinova Gen3 + Robotiq 2F-85
Detail	Rendered from the same calibrated camera viewpoint as the real footage

Step 8 - Paste the robot into the real scene

What: Combine rendered robot + inpainted background into one convincing video.

Technique	Depth-aware compositing - depth decides what's in front of what
Result	Video that looks like the robot did the task, without touching a robot

Step 9 - Split the data

Technique	Stratified train/val/test split at the trial level , not the frame level
Why	Frames from one trial are nearly identical - splitting by frame would leak the test set into training

Step 10 - Train the policy

What: Teach a model to watch video and predict the arm's next movements.

Technique	Diffusion policy - the model learns to turn noise into an action sequence
Vision	ResNet-18 encoders (ImageNet-pretrained)
Conditioning	FiLM - visual features modulate the action decoder
Decoder	1D U-Net over the action sequence
Training	DDPM, 100 noise steps ; Inference: DDIM, 10 steps (faster)
Rotation	6D representation (Zhou et al.) - avoids the wraparound problems of angles
Anti-overfitting	Augmentation, weight decay, early stopping, best-checkpoint selection
Scale	14 trained variants for the ablation study

Step 11 - The contribution: teach the robot to know when it can't see

What: Estimate perception quality every moment, and decide whether to act.

Three signals compared:

Signal	How it works	Quality	Cost
Depth-jump	How much the depth reading jumps between frames	$r = 0.35$	~2 ms
Model uncertainty	Run the model 5x - how much does it disagree with itself?	$r = 0.51$	~340 ms
Combined	Both together	$r = 0.52$	~340 ms

Supporting techniques:

- **Causal temporal depth recovery** - fill in dropped depth pixels using only *past* frames (a live robot can't see the future). Nearly doubled the signal's usefulness.
- **Calibration** - fit the signal against real error on 80% of trials, validate on the other 20%
- **Geometric feasibility check** - lift depth to a 3D point cloud, test whether anything blocks the planned path
- **ACT / DEFER / WAIT state machine** - GO, hold and re-check, or abort if the problem persists

Step 12 - Prove it actually works (offline)

Setup	Held-out trials the model never trained on
Result	11% lower error at a 10-20% pause rate; holds on all five tasks
Detection quality	AUROC 0.795 for spotting bad frames

The controls - this is what makes it convincing:

Control	Result
Pause on random frames	No benefit
Pause on a fixed schedule	No benefit

Same number of frames paused, no improvement. So the gain comes from the robot identifying *which* moments are risky - not from pausing in general.

Statistical methods: paired Wilcoxon signed-rank (non-parametric, right-skewed errors) ; Cohen's d (effect size) ; Holm-Bonferroni (many comparisons) ; McNemar's exact test (binary outcomes) ; bootstrap confidence intervals ; permutation tests ; per-task breakdown always.

Step 13 - Prove it on the real robot (in progress)

Design	Two arms: gate ON vs gate OFF , everything else identical
Ordering	Alternating blocks of ~5, so time-of-day drift can't confound it
Criteria	Success defined per task in advance , in writing
Statistics	Fisher's exact test, Wilson binomial intervals (correct for small samples)
Key trick	Log the gate's decision even when it's switched off - lets us ask "on frames the gate <i>would</i> have paused, did the robot actually fail?"

One technical prerequisite: the model was trained only on *simulated* robot footage, so at deployment we keep showing it a simulated robot - read the real arm's joint angles, render the simulated arm in that pose, composite it over the live camera feed. Skip this and the model sees something it has never encountered.

Step 14 - Report what didn't work

Tried	Outcome
Weighting training by demonstration reliability	Didn't help
Shrinking the model to reduce overfitting	Much worse
Our engineered depth signal vs. simply resampling the model	Engineered signal lost (but is 170x cheaper)

We also found a **bug in our own data**: one input was accidentally identical to the answer the model was meant to predict, making some early results look far better than they were. We found it, removed those results, and documented it.

The one-paragraph version

We teach robots household tasks by filming humans rather than manually guiding the arm. But robots trained this way don't know when their own vision is unreliable - they act confidently regardless. We add a system that continuously estimates how well the robot is seeing and pauses when it isn't. Offline this cuts errors by 11%, and control experiments confirm the gain comes from correctly identifying risky moments rather than from pausing in general. We are now validating it on real hardware.